Intelligent and Communicating Systems (ICS)

LAB N°01 – Introduction to Using and Manipulating a Microcontroller

Full Name: Amel Sadoun – Group 2SQ2

A. THEORY

1. Microcontrollers and Architectures

A microcontroller is a compact integrated circuit designed to perform control functions within an embedded system. It typically integrates a processor (CPU), memory (RAM/ROM/Flash), and input/output peripherals on the same chip.

There are two main architectures:

- **Von Neumann architecture:** Uses a single shared memory for both data and program instructions. The CPU fetches instructions and data using the same bus, which may cause a performance bottleneck.
- **Harvard architecture:** Separates memory for program and data, enabling simultaneous access to both and improving efficiency. Most Arduino boards (AVR and ARM-based) use this architecture.

2. LED and Pushbutton in a Microcontroller

An LED (Light Emitting Diode) serves as an output indicator to show system states or responses. A pushbutton is an input component that triggers actions when pressed. Both must be properly connected to digital I/O pins, using resistors to control current and protect the microcontroller.

3. Getting Familiar with Arduino MKR1010

a) Hardware Aspects: The Arduino MKR1010 features an ARM Cortex-M0+ (SAMD21) microcontroller and a NINA-W102 WiFi module. It runs at 3.3V logic and includes 256 KB of Flash memory and 32 KB of SRAM. It provides digital and analog pins, UART, SPI, and I2C interfaces, making it ideal for IoT applications.

b) Structure of an Arduino Program: An Arduino sketch consists of two main functions:

- `void setup()`: runs once at startup and initializes settings.
- `void loop()`: runs repeatedly, containing the main program logic.

c) Libraries and Functions: Arduino uses libraries to simplify communication with hardware. For example, the `WiFiNINA.h` library enables WiFi control. Common functions include `pinMode()`, `digitalWrite()`, `delay()`, and `WiFi.begin()`.

d) LED and Pushbutton Representation: A digital output pin can send HIGH or LOW signals to control LEDs. An input pin reads HIGH or LOW from a pushbutton. The pin mode must always be configured before use.

4. Arduino Development Tools

a) Installing the IDE and Libraries: The Arduino IDE allows writing, compiling, and uploading sketches via USB. Libraries such as `WiFiNINA` are installed through the Library Manager.

b) Testing and Connecting Components: To verify proper setup, connect the MKR1010 board to a PC, choose the correct COM port, and upload a basic LED blink sketch.

c) Protection of Pins and Components:

- Input pins should use pull-up or pull-down resistors to prevent undefined states.
- Output pins connected to LEDs should use resistors (220–330 Ω) to limit current.
- Pushbuttons should include debouncing mechanisms, either hardware-based or software-based.

5. Microcontroller Simulators

Tools like Tinkercad Circuits, Proteus, and Wokwi simulate Arduino systems. They visualize circuit behavior, serial output, and timing without requiring physical hardware.

B. ACTIVITY

Test 1: Arduino LED Connection

Objective: Connect and control an LED through the digital output of the MKR1010 board. The LED's anode (long leg) is connected to pin 13 via a 220 Ω resistor, and the cathode (short leg) is connected to ground.

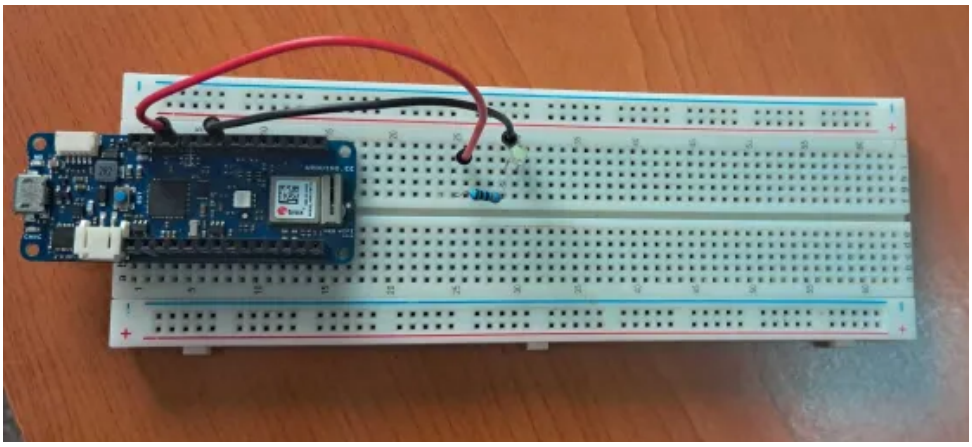

Figure 1 – Simple Arduino MKR1010 LED Connection Circuit

Code Example (Simple LED Blink):

Listing 1: Simple LED blinking code

```
int ledPin = 13; // Built-in LED pin

void setup() {
  pinMode(ledPin, OUTPUT); // Configure pin 13 as output
}

void loop() {
  digitalWrite(ledPin, HIGH); // Turn LED ON
```

```
  delay(2000);                  // Wait 2 seconds
  digitalWrite(ledPin, LOW);    // Turn LED OFF
  delay(2000);                  // Wait 2 seconds
}
```

Test 2: WiFi Network Scanning

Objective: Scan available WiFi networks and light up an LED when the target network is found.

Listing 2: WiFi network scanning code

```
#include <WiFiNINA.h> // Include the WiFiNINA library

int pin = 13; // LED pin

void setup() {
  Serial.begin(9600);  // Start serial communication
  pinMode(pin, OUTPUT); // Set pin 13 as output

  int numSsid = WiFi.scanNetworks(); // Scan available networks
  Serial.print("number of networks found: ");
  Serial.println(numSsid);

  for (int i = 0; i < numSsid; i++) {
    const char* wifiName = WiFi.SSID(i); // Get each network name
    Serial.print(i + 1);
    Serial.print(": Wifi name: ");
    Serial.print(wifiName);
    Serial.print(" | Encryption type: ");
    Serial.println(WiFi.encryptionType(i));

    if (strcmp(wifiName, "AG") == 0) {
      digitalWrite(pin, HIGH);  // Turn LED ON if target found
      delay(3000);
    }
    digitalWrite(pin, LOW);     // Turn LED OFF
    delay(2000);
  }
}

void loop() {
  // Empty loop, runs after setup
}
```

Explanation: This program scans nearby WiFi networks and prints their SSIDs and encryption types on the Serial Monitor. If the network "AG" is detected, the LED turns on for 3 seconds. A video demonstration of this test is available here:
https://drive.google.com/file/d/1W7LEFAdFDzKrOb2n0l4CgAqYjzNIPAIP/view?usp=drive_link

Test 3: WiFi Network Connection

Objective: Connect the Arduino MKR1010 to a WiFi network named "YourDoom" and indicate the connection result using two LEDs.

Code Example (Connecting to WiFi and Indicating Status):

Listing 3: WiFi connection to "YourDoom" network

```
#include <WiFiNINA.h> // Include WiFiNINA library
```

```
const char* s = "YourDoom";    // WiFi network name
const char* p = "ffffffff";    // WiFi password

void setup() {
  pinMode(13, OUTPUT); // LED for success
  pinMode(6, OUTPUT);  // LED for failure

  WiFi.begin(s, p);     // Start WiFi connection
  for (int i = 0; i < 30 && WiFi.status() != WL_CONNECTED; i++) delay(500);

  if (WiFi.status() == WL_CONNECTED) {
    digitalWrite(13, HIGH); // Turn success LED ON
    delay(5000);            // Wait 5 seconds
    WiFi.disconnect();      // Disconnect from WiFi
    digitalWrite(13, LOW);  // Turn success LED OFF
  } else {
    digitalWrite(6, HIGH);  // Turn failure LED ON
  }
}

void loop() {
  // Nothing runs continuously after setup
}
```

Result: The MKR1010 connects to the WiFi network "YourDoom." If successful, the LED on pin 13 lights up for 5 seconds; if it fails, the LED on pin 6 lights up. The Serial Monitor displays each connection stage.

Memory Usage (from compilation output):

```
Sketch uses 15836 bytes (6%) of program storage space. Maximum is 262144 bytes.
Global variables use 3288 bytes (10%) of dynamic memory, leaving 29480 bytes for local variables. Maximum is 32768 bytes.
```

Explanation: The memory report shows that the sketch uses only a small part of the total available storage. This means the program is lightweight, leaving enough space for future features or sensor integrations.

B-3. Conclusion

This lab introduced practical concepts of embedded system programming using the Arduino MKR1010 platform. Through hardware and network experiments, I learned how to:

- Configure and electrically protect digital I/O pins for reliable signal control.
- Interface hardware components such as LEDs and implement wireless communication using the `WiFiNINA.h` library.
- Analyze serial output to monitor program execution and diagnose connection states.
- Evaluate memory allocation to assess program efficiency and resource utilization.

Overall, this experiment strengthened my understanding of microcontroller-based IoT systems, bridging low-level hardware control with network connectivity.